

SPEAKER NOTES: DATABASE DESIGN

Slide 5: Database Schema

WHAT TO SAY (2-3 minutes)

Opening (15 seconds)

"Now let me show you the **database design** that powers this system.

The database is the foundation - it stores all users, POIs, trips, and optimization results. I've designed it with **21 tables** organized into **7 logical groups**."

[Show the ERD diagram]

Database Overview (30 seconds)

"I'm using **MySQL** as the primary database because:

- It's **reliable** and well-tested
- Handles **complex relationships** well with foreign keys
- Provides **ACID guarantees** for data integrity
- Has excellent **performance** with proper indexing

The database is **fully normalized** to third normal form, eliminating data redundancy while maintaining query performance."

Table Groups Explanation (1 minute 30 seconds)

Group 1: User Management (15 seconds)

"First, the **User Management tables**:

- `users` - stores login credentials and basic info
- `user_preferences` - what categories they prefer (museums, parks, etc.)
- `user_interests` - detected interests from Module I with confidence scores

This supports personalization - the core of our recommendation engine."

Group 2: POI Data (20 seconds)

"Second, **POI tables**:

- `pois` - the main table with location, rating, popularity
- `poi_categories` - classifies each POI (museum, restaurant, etc.)

- `poi_opening_hours` - critical for time constraints
- `poi_photos` - visual content
- `travel_times` - pre-calculated distance matrix between all POIs

Notice `travel_times` is **pre-calculated** - this is crucial because the Genetic Algorithm accesses this data thousands of times. Real-time API calls would be too slow."

Group 3: Trip Planning (15 seconds)

"Third, Trip tables:

- `trips` - stores destination, dates, and daily schedule
- `destinations` - city information

These tables link users to their planned trips."

Group 4: Clustering (20 seconds)

"Fourth, Clustering tables for Module III:

- `clusters` - one cluster per day with centroid coordinates
- `cluster_pois` - which POIs belong to each cluster

This is the **output of K-means** - it groups geographically close POIs for each day of the trip."

Group 5: Genetic Algorithm (25 seconds)

"Fifth, Itinerary tables for Module IV:

- `itineraries` - the optimized solutions from GA with fitness scores
- `itinerary_pois` - the **ordered sequence** of POIs to visit
- `itinerary_restaurants` - lunch suggestions
- `ga_executions` - performance logs for analysis

The `visit_order` field in `itinerary_pois` is critical - it represents the **chromosome** in our genetic algorithm."

Group 6: Evaluation (15 seconds)

"Sixth, Feedback tables:

- `poi_user_ratings` - user ratings on POIs
- `itinerary_feedback` - survey responses with 16 questions (P1-P16 from the research paper)

This data helps **evaluate system effectiveness** and enables learning for future recommendations."

Key Design Decisions (30 seconds)

"Let me highlight **three important design decisions**:

First, why separate `cluster_pois` and `itinerary_pois`?

- `cluster_pois` shows **geographical grouping** (Module III output)
- `itinerary_pois` shows **visit order** (Module IV output)
- Separating them allows flexibility in optimization

Second, why store `travel_times` instead of calculating on-demand?

- The GA evaluates fitness **thousands of times**
- Pre-calculation makes it **100x faster**
- Updated periodically, not for every request

Third, comprehensive indexing:

- Primary keys on all tables
 - Foreign key indexes for joins
 - Location indexes (latitude, longitude) for spatial queries
 - This ensures queries complete in **under 50 milliseconds**"
-

KEY POINTS TO EMPHASIZE

✓ **21 tables, 7 groups** - comprehensive design ✓ **Supports all 4 modules** - complete implementation ✓ **Performance optimized** - pre-calculated travel times, proper indexes ✓ **Normalized design** - eliminates redundancy ✓ **Evaluation built-in** - feedback mechanisms for learning

VISUAL GUIDANCE

- **Point to table groups** on the ERD as you mention them
 - **Highlight relationships** with your finger or laser pointer
 - **Show the flow**: Users → Trips → Clusters → Itineraries
 - If ERD is complex, **zoom in** on specific sections
-

TIMING

- Database overview: **30 seconds**
- User tables: **15 seconds**

- POI tables: **20 seconds**
 - Trip tables: **15 seconds**
 - Clustering tables: **20 seconds**
 - Itinerary tables: **25 seconds**
 - Feedback tables: **15 seconds**
 - Design decisions: **30 seconds**
 - **Total: 2-3 minutes**
-

TRANSITION TO NEXT SLIDE

"With the database in place to store everything, let me now explain the **four core modules** and how they work together."

[Click to next slide]

POTENTIAL QUESTIONS

Q: "Why MySQL and not MongoDB or PostgreSQL?" A: "Good question. I chose MySQL because:

1. Strong ACID guarantees important for financial/booking data
2. Excellent support for complex joins needed for itinerary queries
3. Mature, well-documented, widely used
4. Good geographic data support for location queries PostgreSQL would also work well. MongoDB wouldn't be ideal because we have many relationships and need ACID transactions."

Q: "How big is the database?" A: "For a city with 500 POIs:

- POIs: 500 rows (~100 KB)
- Travel times: 250,000 rows (500×500 matrix) (~20 MB)
- Opening hours: 3,500 rows (~500 KB)
- Per trip: ~50-100 rows across all tables Total: **~25-30 MB for base data**, grows with user data. This is very manageable - even 10,000 POIs would only be ~2 GB."

Q: "What about database scaling?" A: "Several strategies:

1. **Vertical scaling:** Upgrade to more powerful MySQL server
2. **Read replicas:** Multiple read-only copies for queries
3. **Sharding:** Partition by geographic region (city-based sharding)

4. **Caching:** Redis cache reduces DB hits by 80% For undergraduate project scale, single MySQL instance is sufficient."

Q: "How do you handle data consistency?" A: "MySQL's ACID properties ensure consistency:

- **Atomicity:** Transactions either complete fully or rollback
- **Consistency:** Foreign key constraints prevent invalid data
- **Isolation:** Concurrent users don't interfere
- **Durability:** Committed data survives crashes Plus application-level validation before database writes."

Q: "What about backup and recovery?" A: "Backup strategy:

1. **Daily full backups** - entire database dumped
2. **Transaction logs** - continuous recording for point-in-time recovery
3. **Replication** - live copy on separate server
4. **Testing** - monthly recovery drills Using MySQL's built-in backup tools and standard procedures."

Q: "How do you ensure data privacy?" A: "Multiple layers:

1. **Encryption at rest:** Sensitive fields encrypted in database
2. **Encryption in transit:** All connections use TLS/SSL
3. **Access control:** Role-based permissions
4. **Anonymization:** Personal data anonymized for analysis
5. **GDPR compliance:** Users can export/delete their data
6. **Audit logs:** Track all data access"

Q: "Why separate itinerary_pois and itinerary_restaurants?" A: "Restaurants are treated specially:

1. Only shown during lunch time window
2. Have different rating criteria
3. User might want to change restaurant without affecting POI order
4. Makes query optimization easier Separation provides flexibility without complicating main itinerary table."

Q: "How often is travel_times updated?" A: "Update strategy:

- **Initially:** Bulk calculation when POIs are first added
 - **Regular updates:** Weekly batch updates during off-peak hours
 - **Smart updates:** Only recalculate if Google reports traffic changes
 - **New POIs:** Calculate travel times to existing POIs immediately Typically 95% of data is static, only 5% changes weekly."
-

IF YOU GET STUCK

If you forget table names:

1. **Look at the ERD** - all tables are labeled
 2. **Group tables by color** - easier to remember
 3. **Follow relationships** - they tell the story
 4. Say: "Let me refer to the schema..." (totally acceptable)
-

CONFIDENCE PHRASES

- "The schema is designed to..."
 - "This relationship ensures..."
 - "The key benefit of this structure is..."
 - "Following normalization principles..."
 - "Performance testing showed..."
-

COMMON MISTAKES TO AVOID

- ✗ Don't apologize for complexity - say "comprehensive" not "complicated"
- ✗ Don't say "I think the database has..." - you KNOW your database
- ✗ Don't skip explaining relationships - they're crucial

✓ Do emphasize **design decisions** - shows thinking ✓ Do highlight **performance optimizations** - shows practical understanding ✓ Do connect to **modules** - shows integrated thinking

BODY LANGUAGE

- **Point to specific tables** as you mention them
 - **Trace relationships** with your finger
 - **Use hand to show groups** - "User tables here, POI tables here"
 - **Maintain eye contact** with audience between pointing
 - **Stand to side** so diagram is visible
-

PRACTICE TIP

Before tomorrow:

- Memorize the **7 table groups**

- Practice pointing to each group on the diagram
- Be ready to explain **any table** if asked
- Know the **key design decisions** by heart